第8章 多　态

“我曾经被问到‘求教，Babbage先生，如果你向机器中输入错误的数字，可以得到正确的答案吗？’我无法恰当地理解产生这种问题的概念上的混淆”

Charles Babbage(1791-1871)

在面向对象的程序设计语言中，多态是继数据抽象和继承之后的第三种基本特征。

多态通过分离做什么和怎么做，从另一角度将接口和实现分离开来。多态不但能够改善代码的组织结构和可读性，还能够创建可扩展的程序——即无论在项目最初创建时还是在需要添加新功能时都可以“生长”的程序。

“封装”通过合并特征和行为来创建新的数据类型。“实现隐藏”则通过将细节“**私有化**”把接口和实现分离开来。这种类型的组织机制对那些拥有过程化程序设计背景的人来说，更容易理解。而多态的作用则是消除类型之间的耦合关系。在前一章中我们已经知道，继承允许将对象视为它自己本身的类型或其基类型来加以处理。这种能力极为重要，因为它允许将多种类型（从同一基类导出的）视为同一类型来处理，而同一份代码也就可以毫无差别地运行在这些不同类型之上了。多态方法调用允许一种类型表现出与其他相似类型之间的区别，只要它们都是从同一基类导出而来的。这种区别是根据方法行为的不同而表示出来的，虽然这些方法都可以通过同一个基类来调用。

在本章中，通过一些基本、简单的例子（这些例子中所有与多态无关的代码都被删掉，只剩下与多态有关的部分），深入浅出地介绍多态（也称作动态绑定、后期绑定或运行时绑定）。

8.1 再论向上转型

在第7章中我们已经知道，对象既可以作为它自己本身的类型使用，也可以作为它的基类型使用。而这种把对某个对象的引用视为对其基类型的引用的做法被称作向上转型——因为在继承树的画法中，基类是放置在上方的。

但是，这样做也有一个问题，具体看下面这个有关乐器的例子。

首先，既然几个例子都要演奏乐符（**Note**），我们就应该在包中单独创建一个**Note**类。

```
//: polymorphism/music/Note.java
// Notes to play on musical instruments.
package polymorphism.music;

public enum Note {
    MIDDLE_C, C_SHARP, B_FLAT; // Etc.
} ///:~
```

enum在第5章中介绍过。

在这里，**Wind**是一种**Instrument**，因此可以从**Instrument**类继承。

```
//: polymorphism/music/Instrument.java
package polymorphism.music;
import static net.mindview.util.Print.*;

class Instrument {
  public void play(Note n) {
    print("Instrument.play()");
  }
```

```
}
 ///:~

//: polymorphism/music/Wind.java
package polymorphism.music;

// Wind objects are instruments
// because they have the same interface:
public class Wind extends Instrument {
  // Redefine interface method:
  public void play(Note n) {
    System.out.println("Wind.play() " + n);
  }
} ///:~

//: polymorphism/music/Music.java
// Inheritance & upcasting.
package polymorphism.music;

public class Music {
  public static void tune(Instrument i) {
    // ...
    i.play(Note.MIDDLE_C);
  }
  public static void main(String[] args) {
    Wind flute = new Wind();
    tune(flute); // Upcasting
  }
} /* Output:
Wind.play() MIDDLE_C
*///:~
```

278

Music.tune()方法接受一个**Instrument**引用，同时也接受任何导出自**Instrument**的类。在**main()**方法中，当一个**Wind**引用传递到**tune()**方法时，就会出现这种情况，而不需要任何类型转换。这样做是允许的——因为**Wind**从**Instrument**继承而来，所以**Instrument**的接口必定存在于**Wind**中。从**Wind**向上转型到**Instrument**可能会“缩小”接口，但不会比**Instrument**的全部接口更窄。

8.1.1 忘记对象类型

Music.java看起来似乎有些奇怪。为什么所有人都故意忘记对象的类型呢？在进行向上转型时，就会产生这种情况；并且如果让**tune()**方法直接接受一个**Wind**引用作为自己的参数，似乎会更为直观。但这样引发的一个重要问题是：如果那样做，就需要为系统内**Instrument**的每种类型都编写一个新的**tune()**方法。假设按照这种推理，现在再加入**Stringed**（弦乐）和**Brass**（管乐）这两种**Instrument**（乐器）：

```
//: polymorphism/music/Music2.java
// Overloading instead of upcasting.
package polymorphism.music;
import static net.mindview.util.Print.*;
class Stringed extends Instrument {
  public void play(Note n) {
    print("Stringed.play() " + n);
  }
}

class Brass extends Instrument {
  public void play(Note n) {
    print("Brass.play() " + n);
  }
}

public class Music2 {
```

279

```
  public static void tune(Wind i) {
    i.play(Note.MIDDLE_C);
  }
  public static void tune(Stringed i) {
    i.play(Note.MIDDLE_C);
  }
  public static void tune(Brass i) {
    i.play(Note.MIDDLE_C);
  }
  public static void main(String[] args) {
    Wind flute = new Wind();
    Stringed violin = new Stringed();
    Brass frenchHorn = new Brass();
    tune(flute); // No upcasting
    tune(violin);
    tune(frenchHorn);
  }
} /* Output:
Wind.play() MIDDLE_C
Stringed.play() MIDDLE_C
Brass.play() MIDDLE_C
*///:~
```

这样做行得通，但有一个主要缺点：必须为添加的每一个新**Instrument**类编写特定类型的方法。这意味着在开始时就需要更多的编程，这也意味着如果以后想添加类似**tune()**的新方法，或者添加自**Instrument**导出的新类，仍需要做大量的工作。此外，如果我们忘记重载某个方法，编译器不会返回任何错误信息，这样关于类型的整个处理过程就变得难以操纵。

280

如果我们只写这样一个简单方法，它仅接收基类作为参数，而不是那些特殊的导出类。这样做情况会变得更好吗？也就是说，如果我们不管导出类的存在，编写的代码只是与基类打交道，会不会更好呢？

这正是多态所允许的。然而，大多数程序员具有面向过程程序设计的背景，对多态的运作方式可能会有一点迷惑。

练习1：(2) 创建一个**Cycle**类，它具有子类**Unicycle**、**Bicycle**和**Tricycle**。演示每一个类型的实例都可以经由**ride()**方法向上转型为**Cycle**。

8.2 转机

运行这个程序后，我们便会发现**Music.java**的难点所在。**Wind.play()**方法将产生输出结果。这无疑是我们所期望的输出结果，但它看起来似乎又没有什么意义。请观察一下**tune()**方法：

```
public static void tune(Instrument i) {
  // ...
  i.play(Note.MIDDLE_C);
}
```

它接受一个**Instrument**引用。那么在这种情况下，编译器怎样才能知道这个**Instrument**引用指向的是**Wind**对象，而不是**Brass**对象或**Stringed**对象呢？实际上，编译器无法得知。为了深入理解这个问题，有必要研究一下绑定这个话题。

8.2.1 方法调用绑定

将一个方法调用同一个方法主体关联起来被称作绑定。若在程序执行前进行绑定（如果有的话，由编译器和连接程序实现），叫做前期绑定。读者可能以前从来没有听说过这个术语，因为它是面向过程的语言中不需要选择就默认的绑定方式。例如，C只有一种方法调用，那就是前期绑定。

上述程序之所以令人迷惑，主要是因为前期绑定。因为，当编译器只有一个**Instrument**引用时，它无法知道究竟调用哪个方法才对。

解决的办法就是*后期绑定*，它的含义就是在运行时根据对象的类型进行绑定。后期绑定也叫做*动态绑定*或*运行时绑定*。如果一种语言想实现后期绑定，就必须具有某种机制，以便在运行时能判断对象的类型，从而调用恰当的方法。也就是说，编译器一直不知道对象的类型，但是方法调用机制能找到正确的方法体，并加以调用。后期绑定机制随编程语言的不同而有所不同，但是只要想一下就会得知，不管怎样都必须在对象中安置某种“类型信息”。

281

Java中除了**static**方法和**final**方法（**private**方法属于**final**方法）之外，其他所有的方法都是后期绑定。这意味着通常情况下，我们不必判定是否应该进行后期绑定——它会自动发生。

为什么要将某个方法声明为**final**呢？正如前一章提到的那样，它可以防止其他人覆盖该方法。但更重要的一点或许是：这样做可以有效地“关闭”动态绑定，或者说，告诉编译器不需要对其进行动态绑定。这样，编译器就可以为**final**方法调用生成更有效的代码。然而，大多数情况下，这样做对程序的整体性能不会有什么改观。所以，最好根据设计来决定是否使用**final**，而不是出于试图提高性能的目的来使用**final**。

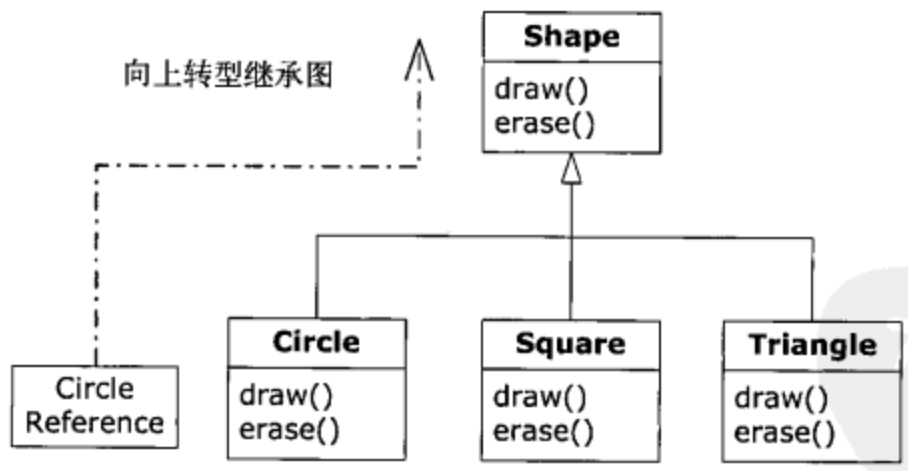
8.2.2 产生正确的行为

一旦知道Java中所有方法都是通过动态绑定实现多态这个事实之后，我们就可以编写只与基类打交道的程序代码了，并且这些代码对所有的导出类都可以正确运行。或者换一种说法，发送消息给某个对象，让该对象去断定应该做什么事。

面向对象程序设计中，有一个经典的例子就是“几何形状”（shape）。因为它很直观，所以经常用到；但不幸的是，它可能使初学者认为面向对象程序设计仅适用于图形化程序设计，实际当然不是这样。

在“几何形状”这个例子中，有一个基类**Shape**，以及多个导出类——如**Circle**、**Square**、**Triangle**等。这个例子之所以好用，是因为我们可以说“圆是一种几何形状”，这种说法也很容易被理解。下面的继承图展示它们之间的关系：

282



向上转型可以像下面这条语句这么简单：

```
Shape s = new Circle();
```

这里，创建了一个**Circle**对象，并把得到的引用立即赋值给**Shape**，这样做看似错误（将一种类型赋值给另一种类型）；但实际上是没问题的，因为通过继承，**Circle**就是一种**Shape**。因此，编译器认可这条语句，也就不会产生错误信息。

假设你调用一个基类方法（它已在导出类中被覆盖）：

```
s.draw();
```

你可能再次认为调用的是**Shape**的**draw()**，因为这毕竟是一个**Shape**引用，那么编译器是怎样知道去做其他的事情呢？由于后期绑定（多态），还是正确调用了**Circle.draw()**方法。

下面的例子稍微有所不同：

```
//: polymorphism/shape/Shape.java
package polymorphism.shape;

public class Shape {
  public void draw() {}
  public void erase() {}
} ///:~
```

283

```
//: polymorphism/shape/Circle.java
package polymorphism.shape;
import static net.mindview.util.Print.*;

public class Circle extends Shape {
  public void draw() { print("Circle.draw()"); }
  public void erase() { print("Circle.erase()"); }
} ///:~
```

```
//: polymorphism/shape/Square.java
package polymorphism.shape;
import static net.mindview.util.Print.*;

public class Square extends Shape {
  public void draw() { print("Square.draw()"); }
  public void erase() { print("Square.erase()"); }
} ///:~
```

```
//: polymorphism/shape/Triangle.java
package polymorphism.shape;
import static net.mindview.util.Print.*;

public class Triangle extends Shape {
  public void draw() { print("Triangle.draw()"); }
  public void erase() { print("Triangle.erase()"); }
} ///:~
```

```
//: polymorphism/shape/RandomShapeGenerator.java
// A "factory" that randomly creates shapes.
package polymorphism.shape;
import java.util.*;

public class RandomShapeGenerator {
  private Random rand = new Random(47);
  public Shape next() {
    switch(rand.nextInt(3)) {
      default:
      case 0: return new Circle();
      case 1: return new Square();
      case 2: return new Triangle();
    }
  }
} ///:~
```

284

```
//: polymorphism/Shapes.java
// Polymorphism in Java.
import polymorphism.shape.*;

public class Shapes {
  private static RandomShapeGenerator gen =
    new RandomShapeGenerator();
  public static void main(String[] args) {
    Shape[] s = new Shape[9];
    // Fill up the array with shapes:
    for(int i = 0; i < s.length; i++)
      s[i] = gen.next();
```

```
    // Make polymorphic method calls:
    for(Shape shp : s)
      shp.draw();
  }
} /* Output:
Triangle.draw()
Triangle.draw()
Square.draw()
Triangle.draw()
Square.draw()
Triangle.draw()
Square.draw()
Triangle.draw()
Circle.draw()
*///:~
```

Shape基类为自它那里继承而来的所有导出类建立了一个公用接口——也就是说，所有形状都可以描绘和擦除。导出类通过覆盖这些定义，来为每种特殊类型的几何形状提供单独的行为。

RandomShapeGenerator是一种“工厂”（factory），在我们每次调用**next()**方法时，它可以为随机选择的**Shape**对象产生一个引用。请注意向上转型是在**return**语句里发生的。每个**return**语句取得一个指向某个**Circle**、**Square**或者**Triangle**的引用，并将其以**Shape**类型从**next()**方法中发送出去。所以无论我们在什么时候调用**next()**方法时，是绝对不可能知道具体类型到底是什么的，因为我们总是只能获得一个通用的**Shape**引用。

main()包含了一个**Shape**引用组成的数组，通过调用**RandomShapeGenerator.next()**来填入数据。此时，我们只知道自己拥有一些**Shape**，除此之外不会知道更具体的情况（编译器也不知道）。然而，当我们遍历这个数组，并为每个数组元素调用**draw()**方法时，与类型有关的特定行为会神奇般地正确发生，我们可以从运行该程序时所产生的输出结果中发现这一点。 285

随机选择几何形状是为了让大家理解：在编译时，编译器不需要获得任何特殊信息就能进行正确的调用。对**draw()**方法的所有调用都是通过动态绑定进行的。

练习2：(1) 在几何图形的示例中添加**@Override**注解。

练习3：(1) 在基类**Shapes.java**中添加一个新方法，用于打印一条消息，但导出类中不要覆盖这个方法。请解释发生了什么。现在，在其中一个导出类中覆盖该方法，而在其他的导出类中不予覆盖，观察又有什么发生。最后，在所有的导出类中覆盖这个方法。

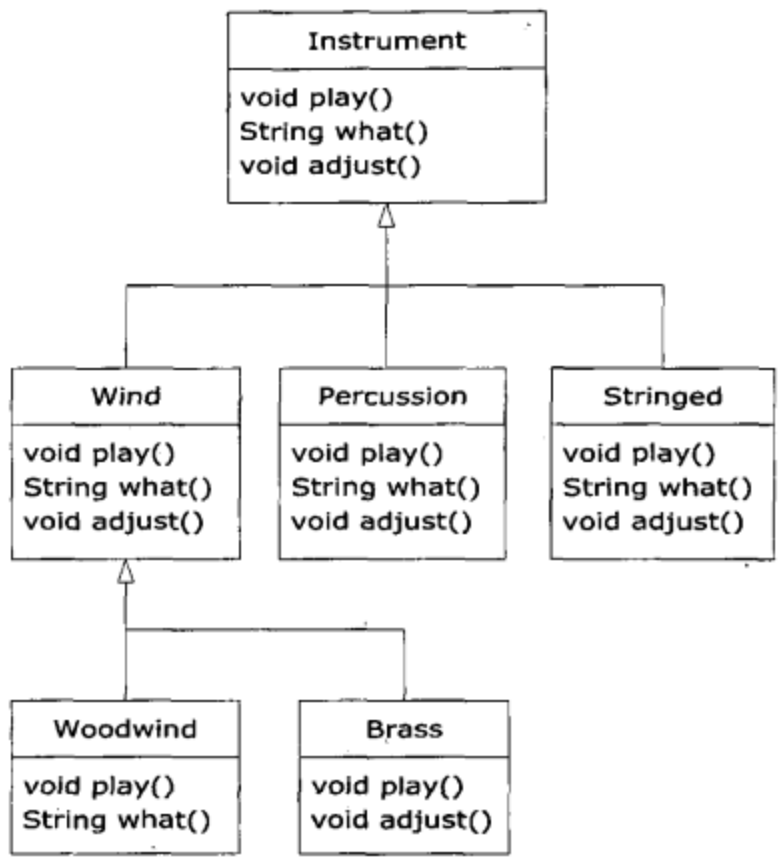
练习4：(2) 向**Shapes.java**中添加一个新的**Shape**类型，并在**main()**方法中验证：多态对新类型的作用是否与在旧类型中的一样。

练习5：(1) 以练习1为基础，在**Cycle**中添加**wheels()**方法，它将返回轮子的数量。修改**ride()**方法，让它调用**wheels()**方法，并验证多态起作用了。

8.2.3 可扩展性

现在，让我们返回到“乐器”（**Instrument**）示例。由于有多态机制，我们可根据自己的需求对系统添加任意多的新类型，而不需更改**tune()**方法。在一个设计良好的OOP程序中，大多数或者所有方法都会遵循**tune()**的模型，而且只与基类接口通信。这样的程序是*可扩展的*，因为可以从通用的基类继承出新的数据类型，从而新添一些功能。那些操纵基类接口的方法不需要任何改动就可以应用于新类。

考虑一下：对于“乐器”的例子，如果我们向基类中添加更多的方法，并加入一些新类，将会出现什么情况呢？请看下图： 286



事实上，不需要改动**tune()**方法，所有的新类都能与原有类一起正确运行。即使**tune()**方法是单独存放在某个文件中，并且在**Instrument**接口中添加了其他的新方法，**tune()**也不需再编译就能正确运行。下面是上图的具体实现：

```
//: polymorphism/music3/Music3.java
// An extensible program.
package polymorphism.music3;
import polymorphism.music.Note;
import static net.mindview.util.Print.*;

class Instrument {
  void play(Note n) { print("Instrument.play() " + n); }
  String what() { return "Instrument"; }
  void adjust() { print("Adjusting Instrument"); }
}

class Wind extends Instrument {
  void play(Note n) { print("Wind.play() " + n); }
  String what() { return "Wind"; }
  void adjust() { print("Adjusting Wind"); }
}

class Percussion extends Instrument {
  void play(Note n) { print("Percussion.play() " + n); }
  String what() { return "Percussion"; }
  void adjust() { print("Adjusting Percussion"); }
}

class Stringed extends Instrument {
  void play(Note n) { print("Stringed.play() " + n); }
  String what() { return "Stringed"; }
  void adjust() { print("Adjusting Stringed"); }
}

class Brass extends Wind {
  void play(Note n) { print("Brass.play() " + n); }
  void adjust() { print("Adjusting Brass"); }
}

class Woodwind extends Wind {
```

287

```
  void play(Note n) { print("Woodwind.play() " + n); }
  String what() { return "Woodwind"; }
}

public class Music3 {
  // Doesn't care about type, so new types
  // added to the system still work right:
  public static void tune(Instrument i) {
    // ...
    i.play(Note.MIDDLE_C);
  }
  public static void tuneAll(Instrument[] e) {
    for(Instrument i : e)
      tune(i);
  }
  public static void main(String[] args) {
    // Upcasting during addition to the array:
    Instrument[] orchestra = {
      new Wind(),
      new Percussion(),
      new Stringed(),
      new Brass(),
      new Woodwind()
    };
    tuneAll(orchestra);
  }
} /* Output:
Wind.play() MIDDLE_C
Percussion.play() MIDDLE_C
Stringed.play() MIDDLE_C
Brass.play() MIDDLE_C
Woodwind.play() MIDDLE_C
*///:~
```

288

新添加的方法**what()**返回一个带有类描述的**String**引用；另一个新添加的方法**adjust()**则提供每种乐器的调音方法。

在**main()**中，当我们将某种引用置入**orchestra**数组中，就会自动向上转型到**Instrument**。

可以看到，**tune()**方法完全可以忽略它周围代码所发生的全部变化，依旧正常运行。这正是我们期望多态所具有的特性。我们所做的代码修改，不会对程序中其他不应受到影响的部分产生破坏。换句话说，多态是一项让程序员“将改变的事物与未变的事物分离开来”的重要技术。

练习6：(1) 修改**Music3.java**，使**what()**方法成为根**Object**的**toString()**方法。试用**System.out.println()**方法打印**Instrument**对象（不用向上转型）。

练习7：(2) 向**Music3.java**添加一个新的类型**Instrument**，并验证多态性是否作用于所添加的新类型。

练习8：(2) 修改**Music3.java**，使其可以像**Shapes.java**中的方式那样随机创建**Instrument**对象。

练习9：(3) 创建**Rodent**（啮齿动物）：**Mouse**（老鼠），**Gerbil**（鼹鼠），**Hamster**（大颊鼠），等等这样一个的继承层次结构。在基类中，提供对所有的**Rodent**都通用的方法，在导出类中，根据特定的**Rodent**类型覆盖这些方法，以便它们执行不同的行为。创建一个**Robent**数组，填充不同的**Rodent**类型，然后调用基类方法，观察发生什么情况。

289

练习10：(3) 创建一个包含两个方法的基类。在第一个方法中可以调用第二个方法。然后产生一个继承自该基类的导出类，且覆盖基类中的第二个方法。为该导出类创建一个对象，将它向上转型到基类型并调用第一个方法，解释发生的情况。

8.2.4 缺陷："覆盖"私有方法

我们试图像下面这样做也是无可厚非的：

```
//: polymorphism/PrivateOverride.java
// Trying to override a private method.
package polymorphism;
import static net.mindview.util.Print.*;

public class PrivateOverride {
  private void f() { print("private f()"); }
  public static void main(String[] args) {
    PrivateOverride po = new Derived();
    po.f();
  }
}

class Derived extends PrivateOverride {
  public void f() { print("public f()"); }
} /* Output:
private f()
*///:~
```

我们所期望的输出是**public f()**，但是由于**private**方法被自动认为是**final**方法，而且对导出类是屏蔽的。因此，在这种情况下，**Derived**类中的**f()**方法就是一个全新的方法；既然基类中的**f()**方法在子类**Derived**中不可见，因此甚至也不能被重载。

结论就是：只有非**private**方法才可以被覆盖；但是还需要密切注意覆盖**private**方法的现象，这时虽然编译器不会报错，但是也不会按照我们所期望的来执行。确切地说，在导出类中，对于基类中的**private**方法，最好采用不同的名字。

8.2.5 缺陷：域与静态方法

一旦你了解了多态机制，可能就会开始认为所有事物都可以多态地发生。然而，只有普通的方法调用可以是多态的。例如，如果你直接访问某个域，这个访问就将在编译期进行解析，就像下面的示例所演示的[⊖]：

290

```
//: polymorphism/FieldAccess.java
// Direct field access is determined at compile time.

class Super {
  public int field = 0;
  public int getField() { return field; }
}

class Sub extends Super {
  public int field = 1;
  public int getField() { return field; }
  public int getSuperField() { return super.field; }
}

public class FieldAccess {
  public static void main(String[] args) {
    Super sup = new Sub(); // Upcast
    System.out.println("sup.field = " + sup.field +
      ", sup.getField() = " + sup.getField());
    Sub sub = new Sub();
    System.out.println("sub.field = " +
      sub.field + ", sub.getField() = " +
      sub.getField() +
      ", sub.getSuperField() = " +
      sub.getSuperField());
```

⊖ 感谢Randy Nichols提出了这个问题。

```
  }
} /* Output:
sup.field = 0, sup.getField() = 1
sub.field = 1, sub.getField() = 1, sub.getSuperField() = 0
*///:~
```

当**Sub**对象转型为**Super**引用时，任何域访问操作都将由编译器解析，因此不是多态的。在本例中，为**Super.field**和**Sub.field**分配了不同的存储空间。这样，**Sub**实际上包含两个称为**field**的域：它自己的和它从**Super**处得到的。然而，在引用**Sub**中的**field**时所产生的默认域并非**Super**版本的**field**域。因此，为了得到**Super.field**，必须显式地指明**super.field**。

291

尽管这看起来好像会成为一个容易令人混淆的问题，但是在实践中，它实际上从来不会发生。首先，你通常会将所有的域都设置成**private**，因此不能直接访问它们，其副作用是只能调用方法来访问。另外，你可能不会对基类中的域和导出类中的域赋予相同的名字，因为这种做法容易令人混淆。

如果某个方法是静态的，它的行为就不具有多态性：

```
//: polymorphism/StaticPolymorphism.java
// Static methods are not polymorphic.

class StaticSuper {
  public static String staticGet() {
    return "Base staticGet()";
  }
  public String dynamicGet() {
    return "Base dynamicGet()";
  }
}

class StaticSub extends StaticSuper {
  public static String staticGet() {
    return "Derived staticGet()";
  }
  public String dynamicGet() {
    return "Derived dynamicGet()";
  }
}

public class StaticPolymorphism {
  public static void main(String[] args) {
    StaticSuper sup = new StaticSub(); // Upcast
    System.out.println(sup.staticGet());
    System.out.println(sup.dynamicGet());
  }
} /* Output:
Base staticGet()
Derived dynamicGet()
*///:~
```

静态方法是与类，而并非与单个的对象相关联的。

292

8.3 构造器和多态

通常，构造器不同于其他种类的方法。涉及到多态时仍是如此。尽管构造器并不具有多态性（它们实际上是**static**方法，只不过该**static**声明是隐式的），但还是非常有必要理解构造器怎样通过多态在复杂的层次结构中运作，这一理解将有助于大家避免一些令人不快的困扰。

8.3.1 构造器的调用顺序

构造器的调用顺序已在第5章进行了简要说明，并在第7章再次提到，但那些都是在多态引入之前介绍的。

基类的构造器总是在导出类的构造过程中被调用，而且按照继承层次逐渐向上链接，以使每个基类的构造器都能得到调用。这样做是有意义的，因为构造器具有一项特殊任务：检查对象是否被正确地构造。导出类只能访问它自己的成员，不能访问基类中的成员（基类成员通常是**private**类型）。只有基类的构造器才具有恰当的知识和权限来对自己的元素进行初始化。因此，必须令所有构造器都得到调用，否则就不可能正确构造完整对象。这正是编译器为什么要强制每个导出类部分都必须调用构造器的原因。在导出类的构造器主体中，如果没有明确指定调用某个基类构造器，它就会“默默”地调用默认构造器。如果不存在默认构造器，编译器就会报错（若某个类没有构造器，编译器会自动合成出一个默认构造器）。

让我们来看下面这个例子，它展示组合、继承以及多态在构建顺序上的作用：

```
//: polymorphism/Sandwich.java
// Order of constructor calls.
package polymorphism;
import static net.mindview.util.Print.*;

class Meal {
  Meal() { print("Meal()"); }
}
class Bread {
  Bread() { print("Bread()"); }
}

class Cheese {
  Cheese() { print("Cheese()"); }
}

class Lettuce {
  Lettuce() { print("Lettuce()"); }
}

class Lunch extends Meal {
  Lunch() { print("Lunch()"); }
}

class PortableLunch extends Lunch {
  PortableLunch() { print("PortableLunch()");}
}

public class Sandwich extends PortableLunch {
  private Bread b = new Bread();
  private Cheese c = new Cheese();
  private Lettuce l = new Lettuce();
  public Sandwich() { print("Sandwich()"); }
  public static void main(String[] args) {
    new Sandwich();
  }
} /* Output:
Meal()
Lunch()
PortableLunch()
Bread()
Cheese()
Lettuce()
Sandwich()
*///:~
```

293

在这个例子中，用其他类创建了一个复杂的类，而且每个类都有一个声明它自己的构造器。其中最重要的类是**Sandwich**，它反映了三层继承（若将自**Object**的隐含继承也算在内，就是四层）以及三个成员对象。当在**main()**里创建一个**Sandwich**对象后，就可以看到输出结果。这也表明了这一复杂对象调用构造器要遵照下面的顺序：

294

1) 调用基类构造器。这个步骤会不断地反复递归下去，首先是构造这种层次结构的根，然后是下一层导出类，等等，直到最低层的导出类。

2) 按声明顺序调用成员的初始化方法。

3) 调用导出类构造器的主体。

构造器的调用顺序是很重要的。当进行继承时，我们已经知道基类的一切，并且可以访问基类中任何声明为**public**和**protected**的成员。这意味着在导出类中，必须假定基类的所有成员都是有效的。一种标准方法是，构造动作一经发生，那么对象所有部分的全体成员都会得到构建。然而，在构造器内部，我们必须确保所要使用的成员都已经构建完毕。为确保这一目的，唯一的办法就是首先调用基类构造器。那么在进入导出类构造器时，在基类中可供我们访问的成员都已得到初始化。此外，知道构造器中的所有成员都有效也是因为，当成员对象在类内进行定义的时候（比如上例中的**b**、**c**和**l**），只要有可能，就应该对它们进行初始化（也就是说，通过组合方法将对象置于类内）。若遵循这一规则，那么就能保证所有基类成员以及当前对象的成员对象都被初始化了。但遗憾的是，这种做法并不适用于所有情况，这一点我们会在下一节中看到。

练习11：(1) 向**Sandwich.java**中添加**Pickle**类。

8.3.2 继承与清理

通过组合和继承方法来创建新类时，永远不必担心对象的清理问题，子对象通常都会留给垃圾回收器进行处理。如果确实遇到清理的问题，那么必须用心为新类创建**dispose()**方法（在这里我选用此名称；读者可以提出更好的）。并且由于继承的缘故，如果我们有其他作为垃圾回收一部分的特殊清理动作，就必须在导出类中覆盖**dispose()**方法。当覆盖被继承类的**dispose()**方法时，务必记住调用基类版本**dispose()**方法；否则，基类的清理动作就不会发生。下例就证明了这一点：

295

```
//: polymorphism/Frog.java
// Cleanup and inheritance.
package polymorphism;
import static net.mindview.util.Print.*;

class Characteristic {
  private String s;
  Characteristic(String s) {
    this.s = s;
    print("Creating Characteristic " + s);
  }
  protected void dispose() {
    print("disposing Characteristic " + s);
  }
}

class Description {
  private String s;
  Description(String s) {
    this.s = s;
    print("Creating Description " + s);
  }
  protected void dispose() {
    print("disposing Description " + s);
  }
}

class LivingCreature {
  private Characteristic p =
    new Characteristic("is alive");
```

```
  private Description t =
    new Description("Basic Living Creature");
  LivingCreature() {
    print("LivingCreature()");
  }
  protected void dispose() {
    print("LivingCreature dispose");
    t.dispose();
    p.dispose();
  }
}

class Animal extends LivingCreature {
  private Characteristic p =
    new Characteristic("has heart");
  private Description t =
    new Description("Animal not Vegetable");
  Animal() { print("Animal()"); }
  protected void dispose() {
    print("Animal dispose");
    t.dispose();
    p.dispose();
    super.dispose();
  }
}

class Amphibian extends Animal {
  private Characteristic p =
    new Characteristic("can live in water");
  private Description t =
    new Description("Both water and land");
  Amphibian() {
    print("Amphibian()");
  }
  protected void dispose() {
    print("Amphibian dispose");
    t.dispose();
    p.dispose();
    super.dispose();
  }
}

public class Frog extends Amphibian {
  private Characteristic p = new Characteristic("Croaks");
  private Description t = new Description("Eats Bugs");
  public Frog() { print("Frog()"); }
  protected void dispose() {
    print("Frog dispose");
    t.dispose();
    p.dispose();
    super.dispose();
  }
  public static void main(String[] args) {
    Frog frog = new Frog();
    print("Bye!");
    frog.dispose();
  }
} /* Output:
Creating Characteristic is alive
Creating Description Basic Living Creature
LivingCreature()
Creating Characteristic has heart
Creating Description Animal not Vegetable
Animal()
Creating Characteristic can live in water
Creating Description Both water and land
```

296

297

```
Amphibian()
Creating Characteristic Croaks
Creating Description Eats Bugs
Frog()
Bye!
Frog dispose
disposing Description Eats Bugs
disposing Characteristic Croaks
Amphibian dispose
disposing Description Both water and land
disposing Characteristic can live in water
Animal dispose
disposing Description Animal not Vegetable
disposing Characteristic has heart
LivingCreature dispose
disposing Description Basic Living Creature
disposing Characteristic is alive
*///:~
```

层次结构中的每个类都包含**Characteristic**和**Description**这两种类型的成员对象，并且它们也必须被销毁。所以万一某个子对象要依赖于其他对象，销毁的顺序应该和初始化顺序相反。对于字段，则意味着与声明的顺序相反（因为字段的初始化是按照声明的顺序进行的）。对于基类（遵循C++中析构函数的形式），应该首先对其导出类进行清理，然后才是基类。这是因为导出类的清理可能会调用基类中的某些方法，所以需要使基类中的构件仍起作用而不应过早地销毁它们。从输出结果可以看到，**Frog**对象的所有部分都是按照创建的逆序进行销毁的。

在这个例子中可以看到，尽管通常不必执行清理工作，但是一旦选择要执行，就必须谨慎和小心。

练习12：(3) 修改练习9，使其能够演示基类和导出类的初始化顺序。然后向基类和导出类中添加成员对象，并说明构建期间初始化发生的顺序。 298

在上面的示例中还应该注意到，**Frog**对象拥有其自己的成员对象。**Frog**对象创建了它自己的成员对象，并且知道它们应该存活多久（只要**Frog**存活着），因此**Frog**对象知道何时调用**dispose()**去释放其成员对象。然而，如果这些成员对象中存在于其他一个或多个对象共享的情况，问题就变得更加复杂了，你就不能简单地假设你可以调用**dispose()**了。在这种情况下，也许就必需使用引用计数来跟踪仍旧访问着共享对象的对象数量了。下面是相关的代码：

```
//: polymorphism/ReferenceCounting.java
// Cleaning up shared member objects.
import static net.mindview.util.Print.*;

class Shared {
  private int refcount = 0;
  private static long counter = 0;
  private final long id = counter++;
  public Shared() {
    print("Creating " + this);
  }
  public void addRef() { refcount++; }
  protected void dispose() {
    if(--refcount == 0)
      print("Disposing " + this);
  }
  public String toString() { return "Shared " + id; }
}

class Composing {
  private Shared shared;
  private static long counter = 0;
  private final long id = counter++;
  public Composing(Shared shared) {
```

```
    print("Creating " + this);
    this.shared = shared;
    this.shared.addRef();
  }
  protected void dispose() {
    print("disposing " + this);
    shared.dispose();
  }
  public String toString() { return "Composing " + id; }
}

public class ReferenceCounting {
  public static void main(String[] args) {
    Shared shared = new Shared();
    Composing[] composing = { new Composing(shared),
      new Composing(shared), new Composing(shared),
      new Composing(shared), new Composing(shared) };
    for(Composing c : composing)
      c.dispose();
  }
} /* Output:
Creating Shared 0
Creating Composing 0
Creating Composing 1
Creating Composing 2
Creating Composing 3
Creating Composing 4
disposing Composing 0
disposing Composing 1
disposing Composing 2
disposing Composing 3
disposing Composing 4
Disposing Shared 0
*///:~
```

299

static long counter跟踪所创建的**Shared**的实例的数量，还可以为**id**提供数值。**counter**的类型是**long**而不是**int**，这样可以防止溢出（这只是一个良好实践，对于本书中的所有示例，这种计数器不可能发生溢出）。**id**是**final**的，因为我们不希望它的值在对象生命周期中被改变。

在将一个共享对象附着到类上时，必须记住调用**addRef()**，但是**dispose()**方法将跟踪引用数，并决定何时执行清理。使用这种技巧需要加倍地细心，但是如果你正在共享需要清理的对象，那么你就没有太多的选择余地了。

练习13：(3) 在**ReferenceCounting.java**中添加一个**finalize()**方法，用来校验终止条件（查看第5章）。

300

练习14：(4) 修改练习12，使得其某个成员对象变为具有引用计数的共享对象，并证明它可以正确运行。

8.3.3 构造器内部的多态方法的行为

构造器调用的层次结构带来了一个有趣的两难问题。如果在一个构造器的内部调用正在构造的对象的某个动态绑定方法，那会发生什么情况呢？

在一般的方法内部，动态绑定的调用是在运行时才决定的，因为对象无法知道它是属于方法所在的那个类，还是属于那个类的导出类。

如果要调用构造器内部的一个动态绑定方法，就要用到那个方法的被覆盖后的定义。然而，这个调用的效果可能相当难于预料，因为被覆盖的方法在对象被完全构造之前就会被调用。这可能会造成一些难于发现的隐藏错误。

从概念上讲，构造器的工作实际上是创建对象（这并非是一件平常的工作）。在任何构造器

内部，整个对象可能只是部分形成——我们只知道基类对象已经进行初始化。如果构造器只是在构建对象过程中的一个步骤，并且该对象所属的类是从这个构造器所属的类导出的，那么导出部分在当前构造器正在被调用的时刻仍旧是没有被初始化的。然而，一个动态绑定的方法调用却会向外深入到继承层次结构内部，它可以调用导出类里的方法。如果我们是在构造器内部这样做，那么就可能会调用某个方法，而这个方法所操纵的成员可能还未进行初始化——这肯定会招致灾难。

通过下面这个例子，我们会看到问题所在：

```
//: polymorphism/PolyConstructors.java
// Constructors and polymorphism
// don't produce what you might expect.
import static net.mindview.util.Print.*;

class Glyph {
  void draw() { print("Glyph.draw()"); }
  Glyph() {
    print("Glyph() before draw()");
    draw();
    print("Glyph() after draw()");
  }
}

class RoundGlyph extends Glyph {
  private int radius = 1;
  RoundGlyph(int r) {
    radius = r;
    print("RoundGlyph.RoundGlyph(), radius = " + radius);
  }
  void draw() {
    print("RoundGlyph.draw(), radius = " + radius);
  }
}

public class PolyConstructors {
  public static void main(String[] args) {
    new RoundGlyph(5);
  }
} /* Output:
Glyph() before draw()
RoundGlyph.draw(), radius = 0
Glyph() after draw()
RoundGlyph.RoundGlyph(), radius = 5
*///:~
```

Glyph.draw()方法设计为将要被覆盖，这种覆盖是在**RoundGlyph**中发生的。但是**Glyph**构造器会调用这个方法，结果导致了对**RoundGlyph.draw()**的调用，这看起来似乎是我们的目的。但是如果看到输出结果，我们会发现当**Glyph**的构造器调用**draw()**方法时，**radius**不是默认初始值1，而是0。这可能导致在屏幕上只画了一个点，或者根本什么东西都没有；我们只能干瞪眼，并试图找出程序无法运转的原因所在。

前一节讲述的初始化顺序并不十分完整，而这正是解决这一谜题的关键所在。初始化的实际过程是：

1) 在其他任何事物发生之前，将分配给对象的存储空间初始化成二进制的零。

2) 如前所述那样调用基类构造器。此时，调用被覆盖后的**draw()**方法（要在调用**RoundGlyph**构造器之前调用），由于步骤1的缘故，我们此时会发现**radius**的值为0。

3) 按照声明的顺序调用成员的初始化方法。

4) 调用导出类的构造器主体。

这样做有一个优点，那就是所有东西都至少初始化成零（或者是某些特殊数据类型中与“零”等价的值），而不是仅仅留作垃圾。其中包括通过“组合”而嵌入一个类内部的对象引用，其值是**null**。所以如果忘记为该引用进行初始化，就会在运行时出现异常。查看输出结果时，会发现其他所有东西的值都会是零，这通常也正是发现问题的证据。

另一方面，我们应该对这个程序的结果相当震惊。在逻辑方面，我们做的已经十分完美，而它的行为却不可思议地错了，并且编译器也没有报错。（在这种情况下，C++语言会产生更合理的行为。）诸如此类的错误会很容易被人忽略，而且要花很长的时间才能发现。

因此，编写构造器时有一条有效的准则：“用尽可能简单的方法使对象进入正常状态；如果可以的话，避免调用其他方法”。在构造器内唯一能够安全调用的那些方法是基类中的**final**方法（也适用于**private**方法，它们自动属于**final**方法）。这些方法不能被覆盖，因此也就不会出现上述令人惊讶的问题。你可能无法总是能够遵循这条准则，但是应该朝着它努力。

练习15：(2) 在**PolyConstructors.java**中添加一个**RectangularGlyph**，并证明会出现本节所描述的问题。

8.4 协变返回类型

Java SE5中添加了协变返回类型，它表示在导出类中的被覆盖方法可以返回基类方法的返回类型的某种导出类型：

```
//: polymorphism/CovariantReturn.java

class Grain {
  public String toString() { return "Grain"; }
}

class Wheat extends Grain {
  public String toString() { return "Wheat"; }
}

class Mill {
  Grain process() { return new Grain(); }
}

class WheatMill extends Mill {
  Wheat process() { return new Wheat(); }
}

public class CovariantReturn {
  public static void main(String[] args) {
    Mill m = new Mill();
    Grain g = m.process();
    System.out.println(g);
    m = new WheatMill();
    g = m.process();
    System.out.println(g);
  }
} /* Output:
Grain
Wheat
*///:~
```

303

Java SE5与Java较早版本之间的主要差异就是较早的版本将强制**process()**的覆盖版本必须返回**Grain**，而不能返回**Wheat**，尽管**Wheat**是从**Grain**导出的，因而也应该是一种合法的返回类

型。协变返回类型允许返回更具体的**Wheat**类型。

8.5 用继承进行设计

学习了多态之后，看起来似乎所有东西都可以被继承，因为多态是一种如此巧妙的工具。事实上，当我们使用现成的类来建立新类时，如果首先考虑使用继承技术，反倒会加重我们的设计负担，使事情变得不必要地复杂起来。

更好的方式是首先选择“组合”，尤其是不能十分确定应该使用哪一种方式时。组合不会强制我们的程序设计进入继承的层次结构中。而且，组合更加灵活，因为它可以动态选择类型（因此也就选择了行为）；相反，继承在编译时就需要知道确切类型。下面举例说明这一点： 304

```
//: polymorphism/Transmogrify.java
// Dynamically changing the behavior of an object
// via composition (the "State" design pattern).
import static net.mindview.util.Print.*;

class Actor {
  public void act() {}
}

class HappyActor extends Actor {
  public void act() { print("HappyActor"); }
}

class SadActor extends Actor {
  public void act() { print("SadActor"); }
}

class Stage {
  private Actor actor = new HappyActor();
  public void change() { actor = new SadActor(); }
  public void performPlay() { actor.act(); }
}

public class Transmogrify {
  public static void main(String[] args) {
    Stage stage = new Stage();
    stage.performPlay();
    stage.change();
    stage.performPlay();
  }
} /* Output:
HappyActor
SadActor
*///:~
```

在这里，**Stage**对象包含一个对**Actor**的引用，而**Actor**被初始化为**HappyActor**对象。这意味着**performPlay()**会产生某种特殊行为。既然引用在运行时可以与另一个不同的对象重新绑定起来，所以**SadActor**对象的引用可以在**actor**中被替代，然后由**performPlay()**产生的行为也随之改变。这样一来，我们在运行期间获得了动态灵活性（这也称作*状态模式*，请参阅www.MindView.com上的《Thinking in Patterns (with Java)》）。与此相反，我们不能在运行期间决定继承不同的对象，因为它要求在编译期间完全确定下来。 305

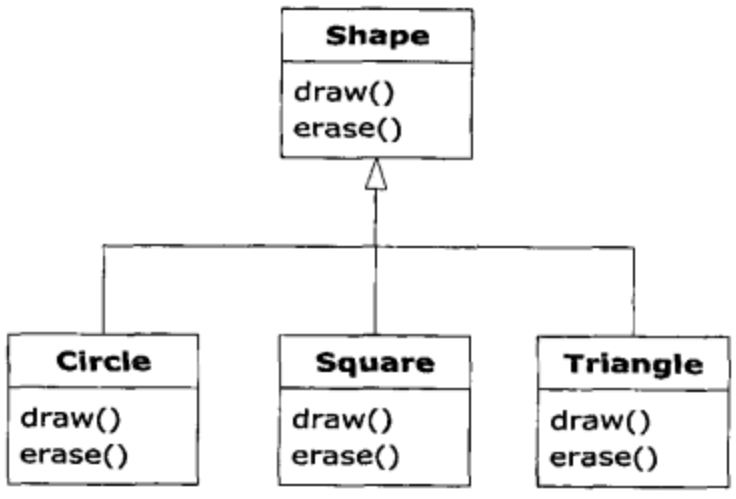
一条通用的准则是：“用继承表达行为间的差异，并用字段表达状态上的变化”。在上述例子中，两者都用到了：通过继承得到了两个不同的类，用于表达**act()**方法的差异；而**Stage**通过运用组合使自己的状态发生变化。在这种情况下，这种状态的改变也就产生了行为的改变。

练习16：(3) 遵循**Transmogrify.java**这个例子，创建一个**Starship**类，包含一个**AlertStatus**

引用，此引用可以指示三种不同的状态。纳入一些可以改变这些状态的方法。

8.5.1 纯继承与扩展

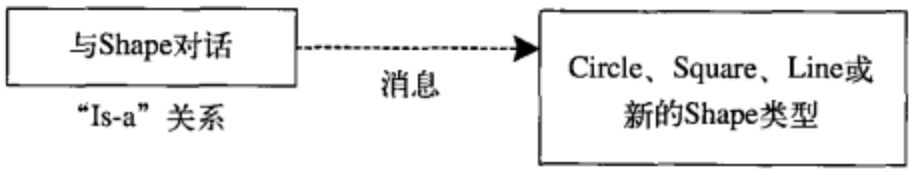
采取“纯粹”的方式来创建继承层次结构似乎是最好的方式。也就是说，只有在基类中已经建立的方法才可以在导出类中被覆盖，如下图所示：



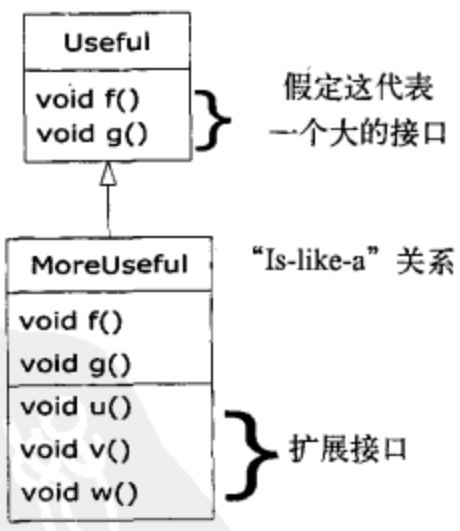
这被称作是纯粹的“is-a”（是一种）关系，因为一个类的接口已经确定了它应该是什么。继承可以确保所有的导出类具有基类的接口，且绝对不会少。按上图那么做，导出类也将具有和基类一样的接口。

306

也可以认为这是一种纯替代，因为导出类可以完全代替基类，而在使用它们时，完全不需要知道关于子类的任何额外信息：



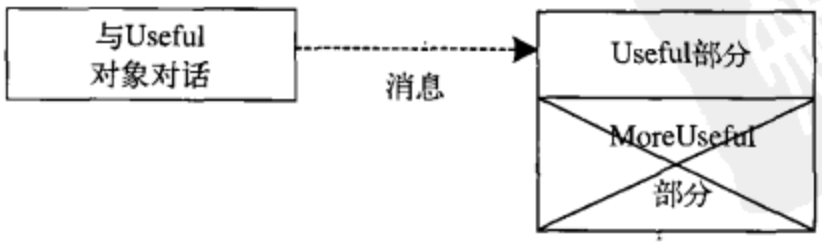
也就是说，基类可以接收发送给导出类的任何消息，因为二者有着完全相同的接口。我们只需从导出类向上转型，永远不需知道正在处理的对象的确切类型。所有这一切，都是通过多态来处理的（如右图所示）。



按这种方式考虑，似乎只有纯粹的is-a关系才是唯一明智的做法，而所有其他的设计都只会导致混乱和注定会失败。这其实也是一个陷阱，因为只要开始考虑，就会转向，并发现扩展接口（遗憾的是，**extends**关键字似乎在怂恿我们这样做）才是解决特定问题的完美方案。这可以称为“is-like-a”（像一个）关系，因为导出类就像是一个基类——它有着相同的基本接口，但是它还具有由额外方法实现的其他特性。

307

虽然这是一种有用且明智的方法（依赖于具体情况），但是它也有缺点。导出类中接口的扩展部分不能被基类访问，因此，一旦我们向上转型，就不能调用那些新方法：



在这种情况下，如果我们不进行向上转型，这样的问题也就不会出现。但是通常情况下，我们需要重新查明对象的确切类型，以便能够访问该类型所扩充的方法。下一节将说明如何做到这一点。

8.5.2 向下转型与运行时类型识别

由于向上转型（在继承层次中向上移动）会丢失具体的类型信息，所以我们就想，通过向下转型——也就是在继承层次中向下移动——应该能够获取类型信息。然而，我们知道向上转型是安全的，因为基类不会具有大于导出类的接口。因此，我们通过基类接口发送的消息保证都能被接受。但是对于向下转型，例如，我们无法知道一个“几何形状”它确实就是一个“圆”，它可以是一个三角形、正方形或其他一些类型。

要解决这个问题，必须有某种方法来确保向下转型的正确性，使我们不致于贸然转型到一种错误类型，进而发出该对象无法接受的消息。这样做是极其不安全的。

在某些程序设计语言（如C++）中，我们必须执行一个特殊的操作来获得安全的向下转型。但是在Java语言中，所有转型都会得到检查！所以即使我们只是进行一次普通的加括弧形式的类型转换，在进入运行期时仍然会对其进行检查，以便保证它的确是我们希望的那种类型。如果不是，就会返回一个**ClassCastException**（类转型异常）。这种在运行期间对类型进行检查的行为称作“运行时类型识别”（RTTI）。下面的例子说明RTTI的行为：

308

```
//: polymorphism/RTTI.java
// Downcasting & Runtime type information (RTTI).
// {ThrowsException}

class Useful {
  public void f() {}
  public void g() {}
}

class MoreUseful extends Useful {
  public void f() {}
  public void g() {}
  public void u() {}
  public void v() {}
  public void w() {}
}

public class RTTI {
  public static void main(String[] args) {
    Useful[] x = {
      new Useful(),
      new MoreUseful()
    };
    x[0].f();
    x[1].g();
    // Compile time: method not found in Useful:
    //! x[1].u();
    ((MoreUseful)x[1]).u(); // Downcast/RTTI
    ((MoreUseful)x[0]).u(); // Exception thrown
  }
} ///:~
```

正如前一个示意图中所示，**MoreUseful**（更有用的）接口扩展了**Useful**（有用的）接口，但是由于它是继承而来的，所以它也可以向上转型到**Useful**类型。我们在**main()**方法中对数组**x**进行初始化时可以看到这种情况的发生。既然数组中的两个对象都属于**Useful**类，所以我们可以调用**f()**和**g()**这两个方法。如果我们试图调用**u()**方法（它只存在于**MoreUseful**），就会返回一条编译时出错消息。

如果想访问**MoreUseful**对象的扩展接口，就可以尝试进行向下转型。如果所转类型是正确的类型，那么转型成功；否则，就会返回一个**ClassCastException**异常。我们不必为这个异常编写任何特殊的代码，因为它指出的是程序员在程序中任何地方都可能会犯的错误。**{ThrowsException}**注释标签告知本书的构建系统：在运行该程序时，预期抛出一个异常。

309

RTTI的内容不仅仅包括转型处理。例如它还提供一种方法，使你可以在试图向下转型之前，查看你所要处理的类型。第14章专门讨论Java运行时类型识别的所有不同方面。

练习17：(2) 使用练习1中的**Cycle**的层次结构，在**Unicycle**和**Bicycle**中添加**balance()**方法，而**Tricycle**中不添加。创建所有这三种类型的实例，并将它们向上转型为**Cycle**数组。在该数组的每一个元素上都尝试调用**balance()**，并观察结果。然后将它们向下转型，再次调用**balance()**，并观察将所产生什么。

8.6 总结

多态意味着“不同的形式”。在面向对象的程序设计中，我们持有从基类继承而来的相同接口，以及使用该接口的不同形式：不同版本的动态绑定方法。

在本章中我们已经知道，如果不运用数据抽象和继承，就不可能理解或者甚至不可能创建多态的例子。多态是一种不能单独来看待的特性（例如，像**switch**语句是可以的），相反它只能作为类关系“全景”中的一部分，与其他特性协同工作。

为了在自己的程序中有效地运用多态乃至面向对象的技术，必须扩展自己的编程视野，使其不仅包括个别类的成员和消息，而且还要包括类与类之间的共同特性以及它们之间的关系。尽管这需要极大的努力，但是这样做是非常值得的，因为它可以带来很多成效：更快的程序开发过程、更好的代码组织、更好扩展的程序以及更容易的代码维护等。

所选习题的答案都可以在名为The Thinking in Java Annotated Solution Guide的电子文档中找到，读者可以从www.MindView.net处购买此文档。

310